24 de Febrero del 2026

Integrantes:
García Galindo Melissa
Martínez Molina Karen
Diego Zarco Ramírez

Tarea 1 - Programación

1. Editar el documento LISTO

https://share.google/mQHyvApJdD3FETdxA

2. Paradigmas de programación e IA:

Paradigma imperativo :
Lenguaje de programación: C
Prompt: Genera un código simple para conocer C
Código:

```
#include <stdio.h>

int main() {
    char nombre[50];
    int edad;

    printf("¡Hola! Bienvenido a tu primer programa en C 😊\n");

    // Pedimos el nombre
    printf("¿Cuál es tu nombre? ");
    scanf("%s", nombre);

    // Pedimos la edad
    printf("¿Cuántos años tienes? ");
    scanf("%d", &edad);

    // Mostramos información
    printf("\nHola, %s\n", nombre);
    printf("Tienes %d años.\n", edad);

    // Condicional
    if (edad >= 18) {
        printf("Eres mayor de edad 🎉\n");
    } else {
        printf("Eres menor de edad 👶\n");
    }

    return 0;
}
```

Paradigma orientado a objetos:
Lenguaje de programación: Python
Prompt: Genera un código simple para conocer Python
Código:

```
# Programa básico en Python

print("¡Hola! Bienvenido a tu primer programa en Python 😊")

# Pedimos el nombre al usuario
nombre = input("¿Cuál es tu nombre? ")

# Pedimos la edad
edad = int(input("¿Cuántos años tienes? "))

# Calculamos el año en que cumplirá 100 años
anio_actual = 2026
anio_100 = anio_actual + (100 - edad)

# Mostramos resultados
print("\nHola,", nombre)
print("Tienes", edad, "años.")
print("Cumplirás 100 años en el año", anio_100)

# Condicional simple
if edad >= 18:
    print("Eres mayor de edad 🎉")
else:
    print("Eres menor de edad 👶")
```

Paradigma funcional:
Lenguaje de programación: Haskell
Prompt: Genera un código simple para conocer Haskell
Código:

```
-- Programa básico en Haskell

main :: IO ()
main = do
    putStrLn "¡Hola! Bienvenido a tu primer programa en Haskell 😊"

    putStr "¿Cuál es tu nombre? "
    nombre <- getLine

    putStr "¿Cuántos años tienes? "
    edadTexto <- getLine
    let edad = read edadTexto :: Int
```

```
putStrLn ("\nHola, " ++ nombre)
putStrLn ("Tienes " ++ show edad ++ " años.")

if edad >= 18
   then putStrLn "Eres mayor de edad 🎉"
   else putStrLn "Eres menor de edad 👶"
```

Paradigma lógico:
Lenguaje de programación: Prolog
Prompt: Genera un código simple para conocer Prolog
Código:

```
% Hechos (cosas que son verdaderas)

padre(juan, pedro).
padre(juan, maria).
madre(ana, pedro).
madre(ana, maria).

% Regla (definimos qué es un hijo)
hijo(X, Y) :- padre(Y, X).
hijo(X, Y) :- madre(Y, X).

% Regla para hermanos
hermano(X, Y) :-
   padre(P, X),
   padre(P, Y),
   X \= Y.
```

Observaciones:

En Python, C y Haskell el código muestra texto y pide información escrita de alguna forma, los tres se parecen en este sentido. Haskell usa notación de suma para trabajar con diferente información, aún sin ser numérica, mientras que Python y C no. Los tres lenguajes también usan lógica condicional. En general diría que los tres estructuran el código de forma parecida. Por otro lado, Prolog usa menos carácteres para su código. Reconoce, por ejemplo, ser padre, madre o hijo como funciones que dependen de dos variables, que no conmutan. Resuelve un problema distinto pero de forma muy directa, con operaciones, de nuevo, basadas en dos variables (X e Y)

3. Ejercicio de casa de cambio.

Tenemos el objeto: Moneda.

Atributos:

divisa: str	por ejemplo, MXN
valor: int	por ejemplo, 5
pais: str	por ejemplo, Mexico
valor_en_dolar_actual: float	por ejemplo, 0.29
valor_en_dolar_inicio_de_anio: float	por ejemplo, 0.26
peso_gr: float	por ejemplo, 7.07
inventario: int	por ejemplo, 7843
color: str	por ejemplo, plateado_y_dorado

Responsabilidades:

entrar_inventario
salir_inventario
mover_posicion_inventario
sumar
convertir_a_euros
convertir_a_yenes

4. Problema del granjero:

zorro

atributos:

posicion_actual: str	
veces_transladado: int	
problemas_con: str	(en este caso, pato)
translaciones_restantes: int	(cuántas veces tiene que cruzar)
posicion_final: bool	(si llegó a su posición final)

pato

atributos:
posicion_actual: str

veces_transladado: int
problemas_con: str (en este caso, maiz y zorro)
translaciones_restantes: int (cuántas veces tiene que cruzar)
posicion_final: bool (si llegó a su posición final)

granjero

atributos:
posicion_actual: str
veces_transladado: int
problemas_con: str (en este caso, nadie)
translaciones_restantes: int (cuántas veces tiene que cruzar)
posicion_final: bool (si llegó a su posición final)

maiz

atributos:
posicion_actual: str
veces_transladado: int
problemas_con: str (en este caso, pato)
translaciones_restantes: int (cuántas veces tiene que cruzar)
posicion_final: bool (si llegó a su posición final)

Solución considerando restricciones:

1-.Llevar al pato
2-.Regresar sólo
3-. Llevar al maíz
4-.Dejar el maíz y regresar con el pato
5-.Dejar al pato y llevar al zorro
6-. regresar sólo
7-.Llevar al pato.

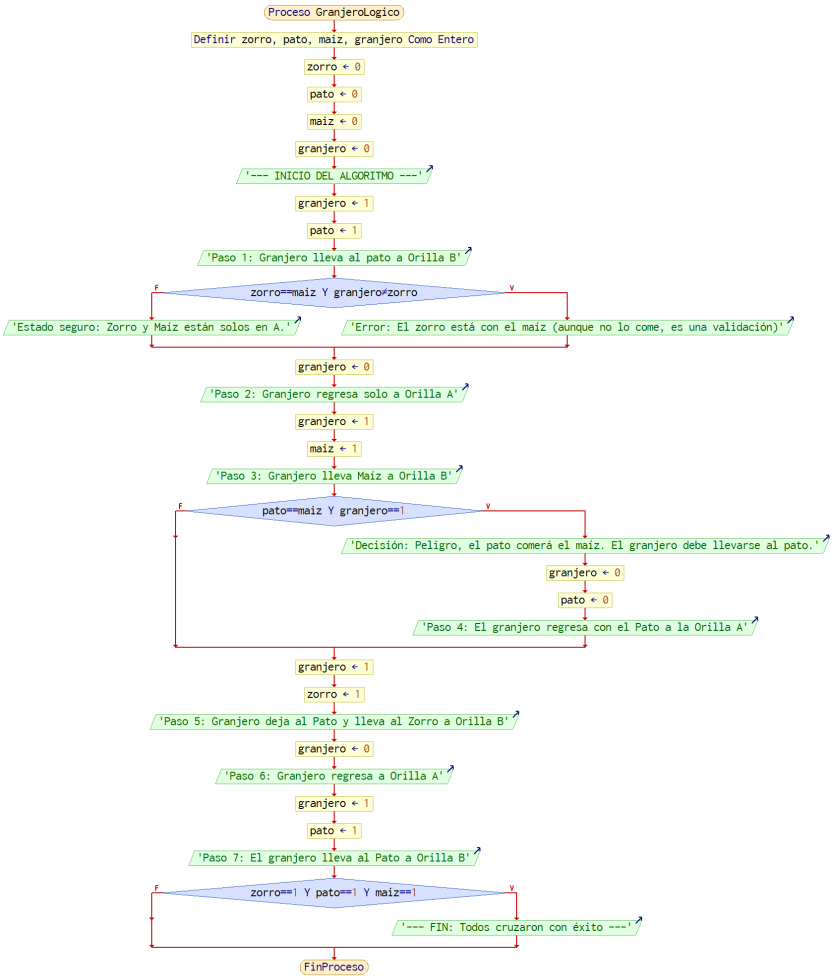
5. Diagrama de flujo con problema del granjero.

La imagen se encuentra debajo del "Código utilizado para hacer diagramas de flujo en Pseint:" Igual adjunto link de la imagen por si no se logra ver claramente:
https://drive.google.com/file/d/1zSuA7b5oeo5F_OLIvUVBfbs3Tf-Nze8K/view?usp=drive_link

Código utilizado para crear diagrama de flujo en Pseint:

```
Proceso GranjeroLogico
    // Definimos posiciones: 0 para Orilla A, 1 para Orilla B
    Definir zorro, pato, maiz, granjero Como Entero;
    zorro <- 0;
    pato <- 0;
    maiz <- 0;
    granjero <- 0;
    Escribir '--- INICIO DEL ALGORITMO ---';
    // PASO 1: Llevar al pato
    granjero <- 1;
    pato <- 1;
    Escribir 'Paso 1: Granjero lleva al pato a Orilla B';
    // VALIDACIÓN DE RESTRICCIÓN
    Si zorro==maiz Y granjero<>zorro Entonces
```

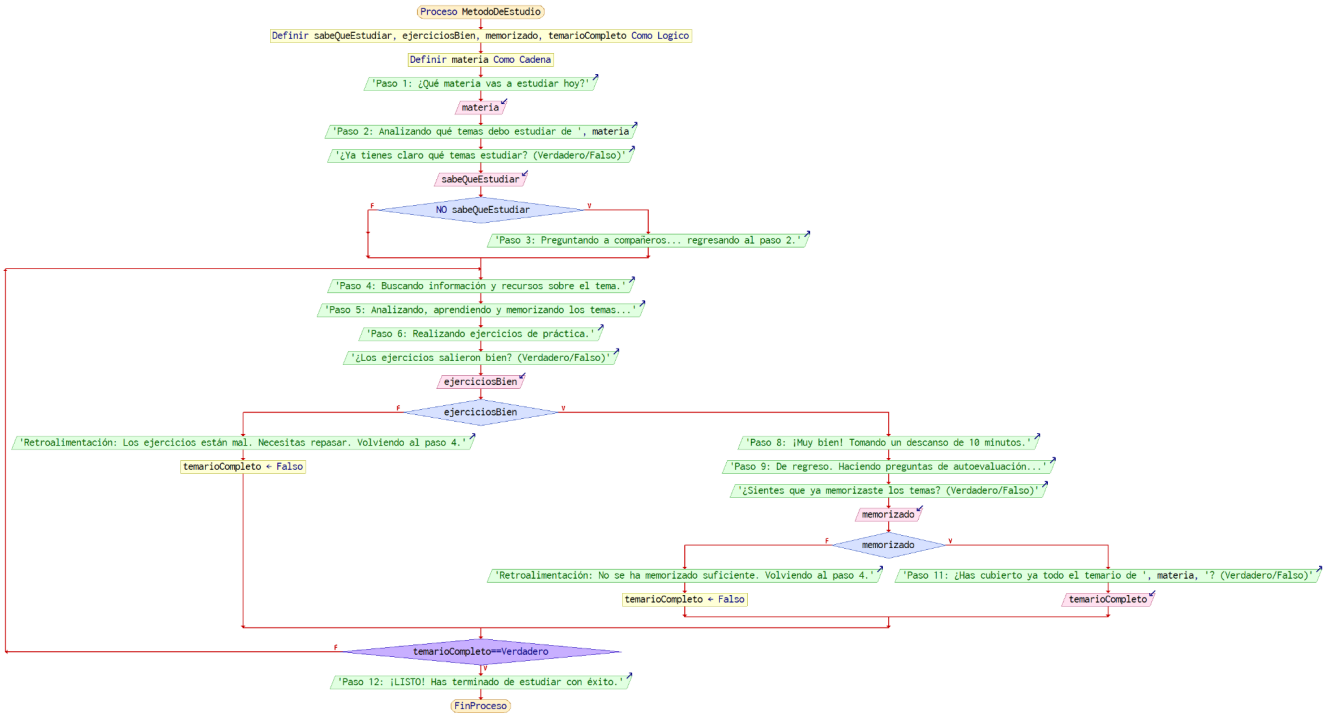
```
        Escribir 'Error: El zorro está con el maíz (aunque no lo come, es una validación)';
    SiNo
        Escribir 'Estado seguro: Zorro y Maíz están solos en A.';
    FinSi
    // PASO 2: Regresar solo
    granjero <- 0;
    Escribir 'Paso 2: Granjero regresa solo a Orilla A';
    // PASO 3: Llevar Maíz
    granjero <- 1;
    maiz <- 1;
    Escribir 'Paso 3: Granjero lleva Maíz a Orilla B';
    // DECISIÓN LÓGICA: ¿Debe regresar con el pato?
    Si pato==maiz Y granjero==1 Entonces
        Escribir 'Decisión: Peligro, el pato comerá el maíz. El granjero debe llevarse al pato.';
        // PASO 4: Regresar con pato
        granjero <- 0;
        pato <- 0;
        Escribir 'Paso 4: El granjero regresa con el Pato a la Orilla A';
    FinSi
    // PASO 5: Llevar Zorro
    granjero <- 1;
    zorro <- 1;
    Escribir 'Paso 5: Granjero deja al Pato y lleva al Zorro a Orilla B';
    // PASO 6: Regresar solo
    granjero <- 0;
    Escribir 'Paso 6: Granjero regresa a Orilla A';
    // PASO 7: Llevar Pato
    granjero <- 1;
    pato <- 1;
    Escribir 'Paso 7: El granjero lleva al Pato a Orilla B';
    // VALIDACIÓN FINAL
    Si zorro==1 Y pato==1 Y maiz==1 Entonces
        Escribir '--- FIN: Todos cruzaron con éxito ---';
    FinSi
FinProceso
```



6. Diagrama de flujo: mi método de estudio.

La imagen se encuentra debajo del "Código utilizado para crear diagrama de flujo en Pseint:" Igual adjunto link de la imagen por si no se logra ver claramente:
https://drive.google.com/file/d/1mdpSmiCj2P4xKTp17utF2Lb9COGG-ITF/view?usp=drive_link

Código utilizado para crear diagrama de flujo en Pseint:

```
Proceso MetodoDeEstudio
	// Definimos las variables para las decisiones (Booleanos)
	Definir sabeQueEstudiar, ejerciciosBien, memorizado, temarioCompleto Como Logico;
	Definir materia Como Cadena;
	// 1. Escojo qué materia quiero estudiar
	Escribir 'Paso 1: ¿Qué materia vas a estudiar hoy?';
	Leer materia;
	// 2. Me pregunto qué debo estudiar
	Escribir 'Paso 2: Analizando qué temas debo estudiar de ', materia;
	// 3. Validación de conocimiento del temario
	Escribir '¿Ya tienes claro qué temas estudiar? (Verdadero/Falso)';
	Leer sabeQueEstudiar;
	Si  NO sabeQueEstudiar Entonces
		Escribir 'Paso 3: Preguntando a compañeros... regresando al paso 2.';
		// Aquí en un diagrama habría una flecha de regreso
	FinSi
	// Iniciamos un ciclo que cubre los pasos del 4 al 11 (Ciclo de Retroalimentación)
	Repetir
		// 4. Buscar la información
		Escribir 'Paso 4: Buscando información y recursos sobre el tema.';
		// 5. Analizo, aprendo y memorizo
		Escribir 'Paso 5: Analizando, aprendiendo y memorizando los temas...';
		// 6. Realizo ejercicios
		Escribir 'Paso 6: Realizando ejercicios de práctica.';
		// 7. Verificar si están bien (Retroalimentación 1)
		Escribir '¿Los ejercicios salieron bien? (Verdadero/Falso)';
		Leer ejerciciosBien;
		Si ejerciciosBien Entonces
			// 8. Tomo un descanso
			Escribir 'Paso 8: ¡Muy bien! Tomando un descanso de 10 minutos.';
			// 9 y 10. Preguntas para ver si memoricé (Retroalimentación 2)
			Escribir 'Paso 9: De regreso. Haciendo preguntas de autoevaluación...';
			Escribir '¿Sientes que ya memorizaste los temas? (Verdadero/Falso)';
			Leer memorizado;
			Si memorizado Entonces
				// 11. ¿He cubierto todo el temario? (Condición final)
				Escribir 'Paso 11: ¿Has cubierto ya todo el temario de ', materia, '?
(Verdadero/Falso)';
				Leer temarioCompleto;
			SiNo
				Escribir 'Retroalimentación: No se ha memorizado suficiente. Volviendo al
paso 4.';
				temarioCompleto <- Falso;
			FinSi
		SiNo
			Escribir 'Retroalimentación: Los ejercicios están mal. Necesitas repasar. Volviendo al
paso 4.';
			temarioCompleto <- Falso;
		FinSi
	Hasta Que temarioCompleto==Verdadero
	// 12. LISTO
```

Escribir 'Paso 12: ¡LISTO! Has terminado de estudiar con éxito.';
FinProceso